

AEM Workflow

Complete Developer Guide

From fundamentals to production-ready Java code

Chapter 1	What is a Workflow?
Chapter 2	Core Concepts & Architecture
Chapter 3	Workflow Step Types
Chapter 4	Creating Workflows in the UI
Chapter 5	Java API Reference
Chapter 6	Launchers & Triggers
Chapter 7	Real-World Example
Chapter 8	Monitoring & Debugging

1

Chapter

What is a Workflow?

Concepts, purpose, and real-world examples

A workflow in AEM is an automated sequence of tasks that runs when something happens — like a page being created, modified, or published. Instead of manually emailing someone to review a page, AEM does it automatically by following a predefined set of steps.

Typical content approval flow:



Why use workflows?

Benefit	Description
Consistent process	Every page follows the same review steps — no one can skip approval
Automatic notifications	AEM notifies the right person at the right time without manual emails
Full audit trail	Every step is recorded — you can see who approved what and when
Custom automation	Run custom Java code — send emails, call external APIs, update JCR nodes
Parallel processing	Multiple reviewers can work simultaneously using AND Split/Join steps

Real-world use cases:

- Content approval — Author writes a page → Manager reviews → Legal approves → Page publishes
- DAM asset processing — Image uploaded → Auto-resize → Generate thumbnails → Move to approved folder
- Translation — Page created → Sent to translation service → Translated page returned → Published
- Scheduled publishing — Page scheduled → Workflow waits until date → Activates at exact time
- User registration — New user registers → Admin review → Welcome email sent → Account activated

Note

AEM ships with many built-in workflows (e.g. DAM Update Asset, Request for Activation). You can also create your own custom workflows from scratch using the Workflow Models console.

2 Chapter

Core Concepts & Architecture

The 5 building blocks every AEM developer must understand

Before building anything, understand these 5 building blocks. Everything in AEM workflows is composed of these concepts.

Workflow Model	The blueprint — defines all the steps and their order. Like a flowchart on paper. You design it once in the AEM UI. Stored at <code>/conf/global/settings/workflow/models</code> . Must be Synced to become active.
Workflow Instance	A live running copy of a model. When you trigger a workflow on a page, AEM creates one instance. Ten pages = ten instances running simultaneously. Stored at <code>/var/workflow/instances</code> .
Payload	The subject of the workflow — what it operates on. Almost always a JCR path like <code>/content/mysite/en/home</code> . Every workflow runs on exactly one payload. Retrieved via <code>item.getWorkflowData().getPayload().toString()</code> .
Work Item	A task assigned to a person. When the workflow reaches a Participant Step, it creates a work item in that person's AEM Inbox. The workflow pauses until they complete or reject it.
Launcher	The trigger that starts a workflow automatically. You configure: when a node under <code>/content/mysite</code> is modified, start workflow X. No manual action needed. Configured at Tools > Workflow > Launchers.

Important JCR paths

Purpose	JCR Path
Editable workflow model	<code>/conf/global/settings/workflow/models/{name}</code>
Runtime model (after Sync)	<code>/var/workflow/models/{name}</code>
All running instances	<code>/var/workflow/instances</code>
Workflow launchers	<code>/conf/global/settings/workflow/launcher</code>
User inbox work items	<code>/var/workflow/instances/{id}/workItems</code>

■ Warning

A model has TWO locations: the editable one under `/conf` and the runtime one under `/var`. You edit in `/conf`. AEM runs from `/var`. You MUST click Sync after every edit, otherwise your changes will not take effect even after saving.

3

Chapter

Workflow Step Types

All built-in step types with configuration and code examples

Step Type	Execution	Use When
Process Step	Automated	Running Java code or ECMA script without human input
Participant Step	Human task	A specific user or group must complete a task in their inbox
Dynamic Participant	Human task	The assignee is calculated at runtime by a script
OR Split	Branching	Workflow must take one of multiple paths based on a condition
AND Split + AND Join	Parallel	Multiple steps must run simultaneously before continuing
Container Step	Grouping	Reuse another workflow model as a sub-workflow
Goto Step	Loop/Jump	Jump back to an earlier step to loop or handle rejection

Process Step — automated code execution

The most commonly used step type. Runs your custom Java class or ECMA script automatically with no human intervention. This is where all your business automation logic goes.

```
@Component(service = WorkflowProcess.class, property = {
    "process.label=Send Email Notification" // name shown in AEM UI dropdown
})
public class SendEmailStep implements WorkflowProcess {

    @Override
    public void execute(WorkItem item, WorkflowSession session,
        MetadataMap args) throws WorkflowException {

        // Get the page/asset path this workflow is running on
        String path = item.getWorkflowData().getPayload().toString();

        // Read arguments configured in the AEM step dialog
        String recipientEmail = args.get("PROCESS_ARGS", "admin@example.com");

        // Your business logic here...
    }
}
```

■ Warning

Always check Handler Advance in the Process Step dialog. Without it, the workflow reaches your step and freezes permanently.

Participant Step — human tasks

Assigns a task to a specific user or group. The workflow pauses until that person opens their AEM Inbox (Tools > Workflow > Inbox) and completes or rejects the task.

- Configure in the AEM UI by entering a user ID (e.g. john) or group name (e.g. content-managers)
- The assignee sees the task in their AEM Inbox at Tools > Workflow > Inbox

- They can Approve, Reject, or Delegate the work item
- Only after they act does the workflow continue to the next step

Dynamic Participant Step — runtime assignee

Same as Participant Step, but the assignee is determined by a script at runtime — useful when you cannot know in advance who should review.

```
// ECMA script – return the username who should receive the task
function getParticipant(workItem, workflowSession, metaDataMap) {
  var payload = workItem.getWorkflowData().getPayload().toString();
  var resource = resourceResolver.getResource(payload);
  // Assign to whoever originally created the page
  return resource.getValueMap().get("jcr:createdBy", "admin");
}
```

OR Split — conditional branching

Evaluates conditions on each outgoing route and follows the first one that returns true. Like an if-else statement in your workflow. Each route has an ECMA script condition.

```
// ECMA condition script for the 'Approved' route
function check(workItem, workflowSession, metaDataMap) {
  var decision = workItem.getMetaDataSet().get("managerDecision", "");
  return "approve".equals(decision);
}
```

4 Chapter

Creating Workflows in the UI

Step-by-step walkthrough from scratch to a working model

1

Open Workflow Models

Click the hammer icon (Tools) in AEM navigation → Workflow → Models. You will see all existing workflow models listed.

2

Create a new model

Click Create (top right) → Create Model. Enter a Title (shown to users, e.g. Content Approval) and a Name (used in the JCR path, e.g. content-approval).

3

Open the model editor

Select your new model from the list and click Edit. The canvas opens with a Start and End node already connected.

4

Add steps from the sidebar

On the left panel you see step categories (Workflow, Forms Workflow, etc.). Drag any step onto the canvas. Connect steps by dragging from the edge of one box to another.

5

Configure each step

Double-click any step to open its dialog. Key fields: Title (display name), Process tab (select your Java class), Arguments (key=value pairs passed to your code), Handler Advance (must be checked for Process Steps).

6

Save the model

Click Save in the toolbar. This saves the editable copy under `/conf/global/settings/workflow/models`.

7

Sync the model — critical!

Click Sync in the toolbar. This publishes the runtime copy to `/var/workflow/models`. Without Sync, your changes will NOT take effect even after saving.

8

Test manually

Go to Sites console → select a page → Properties → Start Workflow → choose your model → Start. Check Tools → Workflow → Instances to see it running.

X Critical

Always check Handler Advance on Process Steps. If unchecked, the workflow reaches that step and stops forever — it will never advance to the next step, and the instance will appear stuck in the Instances console.

5

Chapter

Java API Reference

All 5 core interfaces with every method, parameter and example

The execute() method signature

```
public void execute(
    WorkItem item, // the current work item
    WorkflowSession session, // the workflow engine session
    MetaDataMap args // step config from the AEM UI dialog
) throws WorkflowException { }
```

WorkItem — the current task

Method	Returns	Purpose
getWorkflowData()	WorkflowData	Get the payload (the page/asset path the workflow is running on)
getMetaDataMap()	MetaDataMap	Read/write data shared between steps in the same workflow run
getId()	String	Unique ID of this work item instance — useful for logging
getNode()	WorkflowNode	The current step node — get step title, ID, and type from it
getWorkflow()	Workflow	The parent workflow instance — get initiator, start time, etc.

WorkflowData — the payload

Method	Returns	Purpose
getPayload()	Object	The JCR path — always call .toString() on the result
getPayloadType()	String	Type of payload — almost always "JCR_PATH"
getMetaDataMap()	MetaDataMap	Workflow-level properties like workflowInitiator

MetaDataMap — reading args and sharing data

Method	Purpose
get("PROCESS_ARGS", String.class)	Read step arguments from AEM UI — returns null if missing
get("key", "default")	Read with a fallback default value — safer, never returns null
put("key", value)	Store a value for later steps to read in the same workflow run
containsKey("key")	Check if a key exists before reading — prevents NullPointerExceptions

WorkflowSession — controlling workflow instances

Method	Purpose
startWorkflow(model, data)	Start a new workflow instance programmatically from Java code
getModel("/var/workflow/...")	Load a workflow model by its JCR path — always use /var not /conf
newWorkflowData("JCR_PATH", path)	Create WorkflowData with a payload path — used before startWorkflow

terminateWorkflow(workflow)

Force-stop a running workflow instance — use in error handlers

Complete Java template — copy this

```

@Component(service = WorkflowProcess.class, property = {
    "process.label=My Custom Step" // name in AEM UI dropdown
})
public class MyCustomStep implements WorkflowProcess {

    @Reference
    private ResourceResolverFactory resolverFactory;

    private final Logger log = LoggerFactory.getLogger(getClass());

    @Override
    public void execute(WorkItem item, WorkflowSession session,
        MetadataMap args) throws WorkflowException {

        // 1. Get the payload path (the page/asset being processed)
        String path = item.getWorkflowData().getPayload().toString();
        log.info("[MyStep] Processing: {}", path);

        // 2. Read step config arguments from the AEM dialog
        String myArg = args.get("PROCESS_ARGS", "");

        // 3. Get a ResourceResolver to access the JCR repository
        ResourceResolver resolver = session.adaptTo(ResourceResolver.class);

        try {
            Resource resource = resolver.getResource(path);
            // 4. Your business logic here
            // e.g. modify content, send email, call external API

            // 5. Save JCR changes if you modified any nodes
            resolver.commit();
            log.info("[MyStep] Completed successfully");
        } catch (Exception e) {
            log.error("[MyStep] Failed: {}", e.getMessage(), e);
            throw new WorkflowException("Step failed: " + e.getMessage(), e);
        }
    }
}

```


6 Chapter

Launchers & Triggers

Configure automatic workflow execution based on JCR events

A launcher makes workflows start automatically when something happens in the JCR. No manual trigger needed — AEM monitors the repository and fires the workflow when conditions are met.

Navigate to: Tools > Workflow > Launchers

Launcher configuration fields

Field	What to enter	Example value
Event Type	What JCR event triggers it	Node Created / Node Modified / Node Removed
Node Type	Only fire for this content type	cq:Page or dam:Asset
Path	Only fire for nodes under this path	/content/mysite/en
Condition	Optional ECMA script for filtering	Check a property before firing
Workflow Model	Which model to start	/var/workflow/models/content-approval
Exclude List	Paths to skip even if they match	/content/mysite/temp
Enabled	Toggle on/off without deleting	Checkbox — uncheck to disable temporarily

× Critical

Be careful with the Path field. Setting it to /content with event Node Modified fires on EVERY content change across the entire site — including background system operations. Always scope it to a specific section like /content/mysite/en.

Starting a workflow from Java code

```
// Inside a Sling servlet, OSGi service, or event handler
WorkflowSession wfSession = resourceResolver.adaptTo(WorkflowSession.class);

// Load the model – always use /var path, NOT /conf path
WorkflowModel model = wfSession.getModel("/var/workflow/models/content-approval");

// Create the payload data object
WorkflowData data = wfSession.newWorkflowData("JCR_PATH", "/content/mysite/en/my-page");

// Optionally add metadata before starting
data.getMetadataMap().put("triggeredBy", "my-service");

// Start the workflow
wfSession.startWorkflow(model, data);
```

7

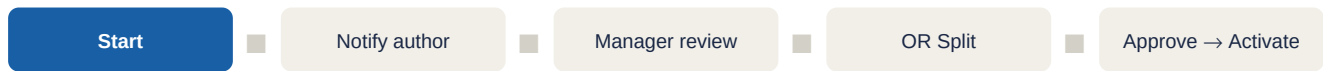
Chapter

Real-World Example

Complete content approval workflow with 4 working Java classes

Building a complete content approval workflow: Author submits a page → Manager reviews → If approved the page activates; if rejected it goes back to the author.

Workflow design



Reject path (from OR Split): → Goto Step → back to Author review

Step 1 — Notify author (Process Step)

```

@Component(service = WorkflowProcess.class, property = {
    "process.label=Notify Author of Submission"
})
public class NotifyAuthorStep implements WorkflowProcess {

    @Override
    public void execute(WorkItem item, WorkflowSession session,
        MetaDataMap args) throws WorkflowException {

        String pagePath = item.getWorkflowData().getPayload().toString();
        String initiator = item.getWorkflow().getInitiator();

        // Store initiator so later steps can use it
        item.getMetaDataMap().put("submittedBy", initiator);

        log.info("Workflow started by {} for {}", initiator, pagePath);
        // Send AEM inbox notification to initiator here
    }
}
  
```

Step 2 — Manager review (Participant Step — configured in UI)

- Drag a Participant Step onto the canvas in the model editor
- Title: Manager Review
- User/Group: content-managers
- Description: Please review and approve or reject this page
- The manager sees the task in Tools > Workflow > Inbox

Step 3 — OR Split condition scripts

```

// ECMA script for the Approved route
function check(workItem, workflowSession, metaDataMap) {
    var decision = workItem.getMetaDataMap().get("managerDecision", "");
    return "approve".equals(decision);
}

// ECMA script for the Rejected route
function check(workItem, workflowSession, metaDataMap) {
    var decision = workItem.getMetaDataMap().get("managerDecision", "");
  
```

```
return "reject".equals(decision);
}
```

Step 4 — Activate the page (Process Step)

```
@Component(service = WorkflowProcess.class, property = {
    "process.label=Activate Approved Page"
})
public class ActivatePageStep implements WorkflowProcess {

    @Reference
    private Replicator replicator;

    @Override
    public void execute(WorkItem item, WorkflowSession session,
        MetadataMap args) throws WorkflowException {

        String path = item.getWorkflowData().getPayload().toString();
        Session jcrSession = session.adaptTo(Session.class);

        try {
            replicator.replicate(jcrSession, ReplicationActionType.ACTIVATE, path);
            log.info("Page activated: {}", path);
        } catch (ReplicationException e) {
            throw new WorkflowException("Activation failed", e);
        }
    }
}
```

8

Chapter

Monitoring & Debugging

Finding problems, fixing stuck workflows, and adding good logging

AEM UI tools for monitoring

UI Location	What you can do
Tools > Workflow > Instances	See all running/completed/failed instances. Terminate or restart them.
Tools > Workflow > Archive	See the history of completed and terminated workflow instances
Tools > Workflow > Failures	See all failed instances with full stack traces and error messages
Tools > Workflow > Inbox	See all pending Participant Step tasks waiting for user action
/system/console/slinglog	Enable debug logging for workflow packages in real time
CRXDE Lite > /var/workflow	Browse the raw JCR nodes for deep inspection of any instance

Common problems and fixes

Workflow gets stuck at a step	Cause: Handler Advance is not checked on a Process Step. Edit the step → check Handler Advance → Save → Sync the model. Fix:
Custom step not in the UI dropdown	Cause: OSGi bundle not deployed or wrong process.label annotation. Check Felix Console > Bundles. Verify @Component annotation has process.label property. Fix:
Changes to the model are not working	Cause: Forgot to click Sync after editing the model. Open the model in the editor → click Sync in the toolbar → wait for confirmation. Fix:
Launcher fires on every tiny change	Cause: Path is too broad or node type is not restricted. Tighten the path to a specific section and add a Condition ECMA script to filter further. Fix:
NullPointerException in execute()	Cause: getPayload() returned an unexpected type or path. Always check getPayloadType().equals("JCR_PATH") before using the payload as a path. Fix:
Work item not appearing in inbox	Cause: Wrong user/group ID in Participant Step config. Verify the exact user ID or group name in the step's configuration dialog. Fix:

Recommended logging pattern

```
private final Logger log = LoggerFactory.getLogger(getClass());

@Override
public void execute(WorkItem item, WorkflowSession session,
MetadataMap args) throws WorkflowException {

String path = item.getWorkflowData().getPayload().toString();
log.info("[MyStep] Starting for payload: {}", path);

try {
// your logic here
log.debug("[MyStep] Processing complete");
} catch (Exception e) {
log.error("[MyStep] Failed for {}: {}", path, e.getMessage(), e);
throw new WorkflowException("Step failed: " + e.getMessage(), e);
}
}
```

Note

View live AEM logs at Tools > Operations > Log Support or at `/system/console/slinglog` in the Felix console. Enable DEBUG level for `com.adobe.granite.workflow` to see detailed workflow engine output.